



Solana Private Channels Web2

Security Assessment

Prepared for
Solana Foundation

Caue Obici

Prepared by
Otter Audits, LLC

Prepared on
September 2nd, 2026

caue@osec.io

Table of Contents

1. Executive Summary	2
1.1. Overview	2
1.2. Key Findings	2
2. Scope	3
3. Findings	4
3.1. Findings summary	4
4. Advisories	5
<u>OS-SPW-ADV-00</u> ● Lack of JWT Revocation	6
<u>OS-SPW-ADV-01</u> ● Wallet-link Challenge Lacks Account Binding	7
<u>OS-SPW-ADV-02</u> ● Failure to Enforce JWT Issuer and Audience	8
<u>OS-SPW-ADV-03</u> ● Grafana Reuses Overprivileged Postgres Credential	9
5. Suggestions	10
<u>OS-SPW-SUG-00</u> ● Unicode Username Validation May Enable Impersonation and Username Check Bypass	11
<u>OS-SPW-SUG-01</u> ● Missing HTTP Timeouts May Allow Resource Exhaustion	12
<u>OS-SPW-SUG-02</u> ● Incorrect Token Account Type Validation May Accept Mint or Multisig Accounts	13
<u>OS-SPW-SUG-03</u> ● Unauthenticated Node Access Relies Solely on Network Isolation	15
A. Vulnerability Rating Scale	16
B. Procedure	17

1. Executive Summary

1.1. Overview

Solana Foundation engaged OtterSec to conduct a security assessment of the `gateway` and `auth` programs within Solana Private Channels and the deployment infrastructure. This assessment was conducted between June 29th and July 7th, 2026. For information on our auditing methodology, refer to [Appendix B](#).

1.2. Key Findings

We produced 8 findings throughout this audit engagement.

Among the most impactful results, we found that the `auth` service embeds a user's role directly into a 24-hour JWT without any revocation mechanism, so an operator who is demoted to user can continue exercising operator-only access and bypass ownership checks on the `gateway` until the stale token expires (● [OS-SPW-ADV-00](#)). We also identified that the wallet-linking challenge is bound only to an opaque user UUID and nonce allowing an attacker to trick a victim wallet into signing the attacker's challenge and register the victim's wallet as "verified" under the attacker's account, granting the attacker read access to the victim's private account data and transaction history through the `gateway` (● [OS-SPW-ADV-01](#)).

Furthermore, the `gateway`'s bearer verification logic builds JWT validation from `Validation::default()` and configures the expected issuer/audience without marking them required, so a token signed with the correct shared secret but missing `iss/aud` claims would still be accepted, weakening the intended token-scoping guarantee to the `auth` service (● [OS-SPW-ADV-02](#)). We also flagged that Grafana's provisioned Postgres datasource reuses the same superadmin credential as the `indexer/operator` services rather than a scoped, read-only role, so any Grafana-side compromise or query abuse could read or exceed what dashboards require (● [OS-SPW-ADV-03](#)).

Additionally, the `gateway`'s token-account ownership check gates only on data length, which does not reliably distinguish real token accounts from other `Token-2022` states, risking `owner/delegate` fields being read from the wrong offsets (● [OS-SPW-SUG-02](#)), and `node` RPC endpoints for the `read/write` nodes have no authentication of their own, relying entirely on Docker network placement for isolation, a risk compounded by shared-network placement of an operator-facing Grafana instance with a history of exploited CVEs (● [OS-SPW-SUG-03](#)).

2. Scope

Gateway is the public JSON-RPC endpoint that routes sendTransaction to the write node and everything else to the read node, optionally enforcing JWT validation, operator-vs-user RBAC on sensitive methods, and per-account ownership checks before serving data.

Auth is the identity service that handles registration/login, issues the JWTs the gateway validates, and lets users prove wallet ownership via signed challenges, with user/operator roles (operator only assignable via DB/admin tooling, never self-service).

The infrastructure scope includes deployment, devnet scripts, docker-compose, and environment configurations.

The source code was delivered to us in a Git repository at

<https://github.com/solana-foundation/solana-private-channels>. This audit was performed against commit [977327b](#).

3. Findings

Findings were rated using the criteria in [Appendix A](#).

We split the findings into **advisories** and **suggestions**. Advisories have an immediate impact and should be remediated as soon as possible. Suggestions do not have an immediate impact but will aid in mitigating future vulnerabilities.

Items are tracked with a stable identifier: **OS-SPW-ADV-*** for advisories and **OS-SPW-SUG-*** for suggestions.

3.1. Findings summary

Severity	Count	
Critical	0	
High	0	
Medium	2	
Low	2	
Informational	4	

4. Advisories

Here, we present a technical analysis of the 4 vulnerabilities we identified during our audit. These vulnerabilities have **immediate** security implications, and we recommend remediation as soon as possible.

ID	Severity	Status	Description
OS-SPW-ADV-00	● Medium	✓ Resolved	Lack of JWT Revocation
OS-SPW-ADV-01	● Medium	✓ Resolved	Wallet-link Challenge Lacks Account Binding
OS-SPW-ADV-02	● Low	○ Ack'd	Failure to Enforce JWT Issuer and Audience
OS-SPW-ADV-03	● Low	○ Ack'd	Grafana Reuses Overprivileged Postgres Credential

Lack of JWT Revocation

ID	Severity	Status
OS-SPW-ADV-00	● Medium	✓ Resolved

Description

The auth service embeds the user role directly into each JWT when a user logs in, and the token lifetime is fixed at 24 hours. The login flow signs the role currently stored on the user record into the token, so subsequent role changes in the database do not alter tokens that have already been issued.

The gateway authorization path validates the bearer token and then makes operator authorization decisions from the decoded `role` claim. For operator-only methods, a token with `role` set to `operator` is allowed to proceed, and for account-gated methods the same operator role bypasses the ownership check entirely. The gateway does not re-read the current user role before granting operator access. The admin role update path changes only the user's database role. Thus, since there is no revocation list, an operator who is demoted to `user` can continue using an already-issued operator JWT until that token expires. During that window, the demoted operator retains the operator read access described by the gateway policy.

Remediation

Bind authorization decisions to current server-side state instead of relying solely on the role claim in long-lived JWTs. For operator-only and operator-bypass decisions, add a revocation mechanism by comparing revoked tokens or reject tokens issued before the user's latest role change.

Patch

Resolved in [0905b22](#).

Wallet-link Challenge Lacks Account Binding

ID	Severity	Status
OS-SPW-ADV-01	● Medium	✓ Resolved

Description

The wallet verification challenge generated by the auth service is bound to the authenticated JWT subject and nonce, but it is not bound to the wallet pubkey that will later be submitted for verification. The challenge response also does not include a human-recognizable account label such as the username. In `auth/src/routes/challenge.rs`, the service builds the message from a static wallet-verification label, the current user's UUID, the challenge nonce, and the expiry timestamp.

`verify_wallet` verifies the supplied signature against the supplied pubkey, and then inserts that pubkey for the JWT subject. Because the signed bytes do not identify the wallet pubkey being linked, the service accepts any submitted pubkey whose private key signed the opaque challenge message for that session. The database schema enforces uniqueness only on the pair of `user_id` and `pubkey`. This prevents duplicate rows for the same user and wallet, but it does not prevent the same wallet pubkey from being registered under different users. Thus, the security boundary is dependent on the signer understanding an opaque UUID. As a result, a signature collected from a wallet owner for another user's challenge can be stored as a verified wallet for that other user.

The impact comes from how the gateway consumes the verified-wallet table. For user-role JWTs, `auth` gates account reads by checking whether the queried system account pubkey, token-account owner, token-account delegate, or DvP participant pubkey appears in `private_channel_auth.verified_wallets` for the JWT subject. The lookup in `db` only checks the caller's `user_id` and the candidate wallet pubkey. As a result, the security boundary is dependent on the signer understanding an opaque UUID.

If an attacker causes a victim wallet to sign the attacker's challenge through a malicious frontend or wallet-connection flow, and registers the victim pubkey under the attacker's account, the gateway may treat the attacker's JWT as authorized for accounts owned or delegated by the victim wallet, since any ATA whose owner/delegate is `victim_pubkey` passes `is_wallet_owned_by_user(attacker_id, victim_pubkey)` check. Thus, allowing the attacker access to private account data and transaction history exposed through the gated read methods.

Remediation

Bind the wallet-link challenge to both the authenticated account and the wallet being verified. Require the client to provide the intended pubkey when requesting the challenge, store that pubkey with the challenge, include it in the signed message. Include a user-facing account identifier, such as the username, in the signed message so the wallet owner can recognize which application account is being linked. This will allow the victim to recognize a malicious message and reject signing it.

Patch

Resolved in [89d33d7](#).

Failure to Enforce JWT Issuer and Audience

ID	Severity	Status
OS-SPW-ADV-02	● Low	○ Ack'd

Description

The gateway authentication boundary is documented and implemented around Bearer JWTs that distinguish operator and user access for gated JSON-RPC methods. In `check_request_auth`, a token accepted by `verify_bearer` is used to authorize operator-only methods or to continue into account ownership checks for user-role requests. The expected issuer and audience constants match the auth service values, and the auth service signs tokens with `iss=private-channel-auth` and `aud=private-channel-gateway`, so the intended contract is that gateway tokens are scoped to this issuer and audience.

However, `verify_bearer` initializes JWT validation from `Validation::default()` and then configures the expected issuer and audience without making those claims required, as shown in `verify_bearer`. That default validation only requires `exp`, so a JWT signed with the same configured secret and carrying otherwise valid gateway claims may be accepted even if it omits `iss` and `aud`. This weakens the documented token-scoping guarantee.

gateway/src/auth.rs

```

349  /// Extracts and verifies a Bearer token from the raw `Authorization` header value.
350  /// Returns `Some(Claims)` on success, `None` if missing, malformed, or expired.
351  fn verify_bearer(auth_header: &str, decoding_key: &DecodingKey) -> Option<Claims> {
352      let token = auth_header.strip_prefix("Bearer ")?;
353      let mut validation = Validation::default();
354      validation.set_issuer(&[JWT_ISSUER]);
355      validation.set_audience(&[JWT_AUDIENCE]);
356      decode::<Claims>(token, decoding_key, &validation)
357          .ok()
358          .map(|data| data.claims)
359  }
```

Gateway JWT verification configures issuer and audience but does not require those claims.

Remediation

Update `verify_bearer` so the JWT validation explicitly requires `exp`, `iss`, and `aud` before calling `decode`.

Patch

Solana Foundation acknowledged this recommendation.

Grafana Reuses Overprivileged Postgres Credential

ID	Severity	Status
OS-SPW-ADV-03	● Low	○ Ack'd

Description

The provisioned Grafana Postgres datasource connects directly to the indexer database using the same `POSTGRES_USER` and `POSTGRES_PASSWORD` variables that initialize `postgres-indexer` and are also used by the indexer and operator services in their `DATABASE_URL` values. This reused credential is the superadmin for the indexer database, so Grafana receives an operational database credential instead of a datasource-specific account with privileges limited to dashboard queries.

Grafana needs database access for visualizations, but using the database owner credential expands the impact of any Grafana user or workflow that can issue or alter datasource-backed queries. That access may allow broader queries over the indexer transaction data than the dashboards require, potentially breaking the privacy guarantees.

Remediation

Create a dedicated Postgres role for Grafana with no ownership or write privileges, scoped to the metric views.

Patch

Solana Foundation acknowledged this recommendation.

5. Suggestions

Here, we present a discussion of the 4 suggestions we identified during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Severity	Status	Description
OS-SPW-SUG-00	● Informational	✓ Resolved	Unicode Username Validation May Enable Impersonation and Username Check Bypass
OS-SPW-SUG-01	● Informational	✓ Resolved	Missing HTTP Timeouts May Allow Resource Exhaustion
OS-SPW-SUG-02	● Informational	✗ To-do	Incorrect Token Account Type Validation May Accept Mint or Multisig Accounts
OS-SPW-SUG-03	● Informational	✗ To-do	Unauthenticated Node Access Relies Solely on Network Isolation

Unicode Username Validation May Enable Impersonation and Username Check Bypass

ID	Severity	Status
OS-SPW-SUG-00	● Informational	✓ Resolved

Description

The auth service registration path validates usernames by iterating over `req.username.chars()` and accepting characters classified as alphanumeric through the function `is_alphanumeric()`, plus underscores and hyphens. In Rust, this character classification is Unicode-aware, so the implementation accepts non-ASCII letters or digits.

Accepted usernames are stored as unique **TEXT** values in the `private_channel_auth.users` table and are later used for exact username lookups during login and by admin tooling for role changes and wallet attachment. Because visually similar Unicode usernames can be distinct database values, an account name may appear similar to another account while bypassing exact username checks that compare the stored string. This could enable username confusion or impersonation-style phishing if operators or users rely on displayed usernames for account identification.

Remediation

Restrict username validation to the intended ASCII character set by using ASCII-specific checks.

Patch

Resolved in [89d33d7](#).

Missing HTTP Timeouts May Allow Resource Exhaustion

ID	Severity	Status
OS-SPW-SUG-01	● Informational	✓ Resolved

Description

The gateway and auth services lack visible HTTP timeout enforcement. In the gateway, `run` binds a `TcpListener` and delegates to `serve`, whose accept loop spawns a new Tokio task for each accepted TCP stream. Each task serves the connection with `http1::Builder::new().serve_connection(...)`, and the surrounding code does not show a connection semaphore, idle timeout, or header read timeout for client connections.

The gateway limits request size with `MAX_BODY_SIZE` and `Limited`, and it rejects oversized `Content-Length` values before reading the body. However, request handling then awaits `limited_body.collect()` directly before JSON parsing, authentication checks, and forwarding to the read or write node. Because that body collection is size-limited but not deadline-limited, a client that slowly streams a body below the configured maximum may keep the socket and spawned task occupied for an extended period.

Remediation

Configure explicit timeout and concurrency controls for all externally-facing HTTP paths. For the gateway, add Hyper HTTP/1 timer configuration such as a Tokio timer and header read timeout on the connection builder, wrap `limited_body.collect()` in a bounded `tokio::time::timeout`.

Patch

Resolved in [dcb0608](#).

Incorrect Token Account Type Validation May Accept Mint or Multisig Accounts

ID	Severity	Status
OS-SPW-SUG-02	● Informational	✗ To-do

Description

`check_account_data_ownership` routes accounts owned by the SPL Token and Token-2022 programs into `check_token_account_ownership`, where the gateway treats any token-program-owned account with at least `TOKEN_ACCOUNT_SIZE` bytes as eligible for token-account ownership checks, as shown in `check_token_account_ownership`. This size-only type check rejects short legacy mint accounts, but it does not explicitly distinguish token accounts from other token-program account states that can also have data lengths at or above 165 bytes.

gateway/src/auth.rs

```

244  /// Ownership check for SPL Token and Token-2022 accounts. Both programs share
245  /// the same base layout, so this checks the `owner` field (bytes 32-63) and,
246  /// if it doesn't match, the `delegate` field (bytes 76-107) when present - a
247  /// delegate has spend authority and counts as ownership for read access.
248  ///
249  /// Mints are owned by the same programs but are only 82 bytes, below
250  /// `TOKEN_ACCOUNT_SIZE`; they are not user accounts and are denied.
251  async fn check_token_account_ownership(
252      data: &[u8],
253      user_id: Uuid,
254      auth_db: &PgPool,
255  ) -> AuthDecision {
256      if data.len() < TOKEN_ACCOUNT_SIZE {
257          return AuthDecision::Reject(StatusCode::FORBIDDEN, forbidden_body());
258      }
259      [...]
260  }
```

Token-program account ownership dispatch and size-only token-account gate

After the length check, the function reads the token-account `owner` field and, if present, the `delegate` field from fixed SPL token-account offsets and compares those public keys against the user's verified wallets. Because Token-2022 mints with extensions and multisig accounts can be larger than 165 bytes, the gateway may parse non-token-account data as though it were a token account and make an authorization decision from offsets that do not represent the expected token-account owner or delegate fields.

Remediation

Replace the size-only token-account type check with explicit SPL Token and Token-2022 account-state validation before reading owner or delegate fields. At minimum, reject token-program data shorter than 165 bytes, reject data whose size is exactly 355 bytes (the multisig

account data size), and for account data longer than 165 bytes inspect the Token-2022 account type extension byte so mint accounts are rejected before fixed token-account offsets are used for authorization.

Unauthenticated Node Access Relies Solely on Network Isolation

ID	Severity	Status
OS-SPW-SUG-03	● Informational	✗ To-do

Description

The private-channel read and write node RPC services do not enforce node-side authentication. The repository documentation states that gateway RBAC is optional and, even when enabled, protects only the gateway port; the read and write node RPC ports have no authentication of their own. The reference Docker Compose deployment limits host exposure by publishing the read and write node ports on `127.0.0.1`, so external access is reduced by deployment topology rather than by authentication at the node RPC layer.

This makes network placement part of the access-control boundary for the node RPC interfaces. In the reference Compose deployment, the read node, write node, gateway, and Grafana service are attached to `private-channel-network`, and Grafana is also published as an operator-facing dashboard. If a service on that shared network is compromised, the node RPC services may be reachable without passing through the gateway's JWT and RBAC checks.

Thus, relying on that deployment topology means any production environment that attaches unrelated or higher-risk services to the same network as the nodes could unintentionally expand access to unauthenticated read or write RPC endpoints. Particularly, there have been instances where Odays and CVEs have been exploited in the grafana service.

Remediation

Document the node RPC trust boundary explicitly and advise the companies to segregate the network and isolate the services that does not need to communicate with the nodes.

Appendix A.

Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the general findings.

Severity	Description
● Critical	Vulnerabilities that immediately result in a loss of availability, consensus divergence, or exploitable memory corruption with minimal preconditions.
● High	Vulnerabilities that may result in consensus divergence or denial of service but require specific transaction or network conditions to exploit.
● Medium	Vulnerabilities that may result in incorrect behavior or state divergence under specific configurations or edge cases.
● Low	Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.
● Informational	Best practices to mitigate future security risks. These are classified as general findings.

Resolution status is one of  Resolved,  Ack'd, or  To-do.

Appendix B.

Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

The design review asks whether the programs's architecture and operating assumptions support its intended behavior. We study the inputs it accepts, the decisions it makes, the information shared between components, and the services or participants it trusts. We then consider how it behaves when those inputs are unexpected or conflicting. The goal is to find conditions that can produce an incorrect decision, inconsistent state, or loss of service.

Design problems often arise from interactions rather than one defective function. Two components may each process a value correctly but interpret it differently. A failure may also leave one component ready to continue while another treats the same operation as incomplete. Local checks can look correct in both cases, so we follow the complete operation across component boundaries.

The implementation review asks whether the code matches the design. We trace untrusted data from its entry point to the point where it affects behavior, checking the assumptions made along the way. The review covers validation, error handling, state updates, concurrency, persistence, resource use, and cleanup. Its exact focus depends on the target's language, runtime, and architecture rather than a fixed list of bug classes.

The boundary between design and implementation is not exact. Missing validation may be a local coding error, while the same omission across several components may reflect an unclear rule. We classify a finding by its root cause, but judge its impact from the resulting behavior and the conditions needed to trigger it.

We begin by reading the target as a team and agreeing on its main components, trust boundaries, and expected behavior. Auditors then review different paths through the system and compare notes where those paths meet. We reproduce suspected issues with focused tests or traces when the setup allows. Before reporting a finding, another auditor reviews its trigger, impact, and proposed remediation.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process